

编译原理大作业 2

类 Rust 语言语义分析与中间代码生成器设计与实现

2351078 李堂辉 2351837 高胜寒 2352197 冉子易

同济大学

(按学号排序)

2026 年 5 月 8 日

目录

1	概述	3
1.1	任务描述	3
1.2	实现语言与工具	3
1.3	已实现的语法规则	4
2	语义分析原理与设计	5
2.1	符号表的设计与管理	5
2.2	静态语义检查规则	5
3	中间代码生成原理与设计	5
3.1	四元式格式与定义	5
3.2	控制流的语法制导翻译	6
4	详细设计	7
4.1	编译器与 IR 生成器详细设计	7
4.2	中间代码模块详细设计	8
4.3	四元式序列节点设计	8
4.4	错误处理	9
5	测试与验证	9
5.1	测试用例设计	9
5.2	语义错误诊断结果示例	9
5.3	中间代码（四元式）生成示例	10

目录	2
6 环境配置与运行指南	10
6.1 环境配置	10
6.2 编译与测试	10
7 心得与思考	11
7.1 关于 LR(1) 无法处理的情况	11
7.2 1.4 形参列表的语言	11
7.3 9.1 元组 vs 8.1 数组的产生式差异	11
8 参考资料	11

1 概述

1.1 任务描述

本项目实现了一个面向类 Rust 语言子集的前端编译分析工具，包括**编译器与 IR 生成器** (Lexer) 和**中间代码模块** (Parser)。该工具能够：

1. 在词法和语法分析的基础上，对抽象语法树（四元式序列）进行深度优先遍历，完成静态语义检查（作用域、类型推断、可变性校验等）；
2. 诊断静态语义错误，对于未声明变量、类型不一致、常量被修改等错误予以精确定位和报错；
3. 对无语义错误的源程序，采用语法制导翻译，生成等价的四元式形式的中间代码 (Intermediate Representation)。

1.2 实现语言与工具

- **实现语言**：Rust
- **词法与语法分析**：手写状态机扫描器与递归下降分析器
- **语义分析方法**：维护多级符号表，自底向上属性综合推导
- **中间代码生成**：采用基于四元式的语法制导翻译与控制流回填
- **构建工具**：Cargo (Rust 标准构建系统)

1.3 已实现的语法规则

表 1: 已实现的语法规则概览

编号	规则名称	说明
0.1	变量属性	<code>mut</code> / 空
0.2	类型	<code>i32</code> , <code>&T</code> , <code>&mut T</code> , <code>[T;N]</code> , <code>(T1,T2)</code>
0.3	左值	标识符、解引用、数组下标、元组下标
1.1	基础程序	程序 $\rightarrow$ 声明串
1.2	语句	语句串 $\rightarrow$ 语句语句串
1.3	返回语句	<code>return</code> [表达式] ;
1.4	函数输入	形参列表
1.5	函数输出	函数返回类型
2.0	变量声明	<code>let</code> [mut] <code>id</code> [:type]
2.1	变量声明语句	<code>let</code> 声明 ;
2.2	赋值语句	左值 = 表达式 ;
2.3	变量声明赋值	<code>let</code> 声明 = 表达式 ;
3.1	基本表达式	整数、标识符、括号表达式
3.2	比较运算	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>==</code> , <code>!=</code>
3.3	加减运算	<code>+</code> , <code>-</code>
3.4	乘除运算	<code>*</code> , <code>/</code>
3.5	函数调用	<code>id(args)</code>
4.1	if 选择	<code>if</code> 表达式块
4.2	if-else	增加 <code>else</code> 分支
4.3	if-else if	增加 <code>else if</code> 分支
5.0	循环语句	<code>while</code> / <code>for</code> / <code>loop</code>
5.1	while 循环	<code>while</code> 表达式块
5.2	for 循环	<code>for</code> 变量 <code>in</code> 可迭代块
5.3	loop 循环	<code>loop</code> 块
5.4	break/continue	循环控制
6.1–6.4	引用与借用	不可变/可变引用、解引用
7.0–7.4	函数表达式块	表达式块、选择表达式、循环表达式
8.1–8.3	数组	数组类型、数组字面量、数组下标
9.1–9.3	元组	元组类型、元组字面量、元组下标

2 语义分析原理与设计

2.1 符号表的设计与管理

语义分析的核心是**符号表**的管理。本项目在 `compiler.rs` 中设计了作用域敏感的符号表系统。为了支持局部作用域和变量重影 (Shadowing)，符号表使用了基于栈的嵌套结构：

```

1 pub struct Compiler {
2     scopes: Vec<HashMap<String, SymbolInfo>>,
3     // ...
4 }
5 struct SymbolInfo {
6     mutable: bool,
7     ty: Type,
8     arg: Arg, // 在中间代码中的临时变量或分配地址
9 }

```

进入函数体、块结构或控制流内部时调用 `push_scope()` 创建新的局部作用域映射；退出时调用 `pop_scope()`。查找变量时采用从栈顶到栈底倒序查找的策略，完美支持了同名变量的就近遮蔽特性。

2.2 静态语义检查规则

我们在遍历抽象语法树（四元式序列）时拦截了多种静态语义错误：

1. **变量未声明报错**：使用标识符前未在任何层级作用域找到对应的 `SymbolInfo`。
2. **不可变变量二次赋值报错**：检查 `is_mutable` 字段。
3. **类型不匹配检查**：对于表达式 `L = R` 或 `L + R`，递归推导出子树类型后必须相等，否则拦截。
4. **控制流结构约束**：`break` 和 `continue` 语句必须在循环体内出现，这通过维护一个 `loop_stack` 栈来实现。

3 中间代码生成原理与设计

3.1 四元式格式与定义

中间代码采用了**四元式** (Quadruple) 的形式，定义如下：

(操作符 (Op)，源操作数 1(Arg1)，源操作数 2(Arg2)，结果 (Result))

在 Rust 代码中被定义为枚举结构,其中 Op 包括 ADD, SUB, ASSIGN, JUMP, JUMP_IF_FALSE, CALL, RETURN, Deref, LOAD_ARRAY 等。操作数可以为空 (-),或者是字面量常量、变量名、临时变量 tN、或标号 LN。

3.2 控制流的语法制导翻译

以 while 循环为例,其语法制导翻译的执行流程如下:

```
1 let start_label = self.new_label();
2 let end_label = self.new_label();
3
4 // 1. 发射循环起始标号
5 self.ir.emit(Op::Label, Arg::Empty, Arg::Empty, Arg::Label(
    start_label));
6
7 // 2. 编译条件表达式
8 let (_, cond_arg) = self.compile_expression(cond);
9
10 // 3. 条件为假则跳出循环
11 self.ir.emit(Op::JumpIfFalse, cond_arg, Arg::Empty, Arg::Label(
    end_label));
12
13 // 4. 将标号压栈供 break/continue 使用
14 self.loop_stack.push((start_label, end_label, None));
15 self.compile_block(body);
16 self.loop_stack.pop();
17
18 // 5. 跳转回起始并标定结束
19 self.ir.emit(Op::Jump, Arg::Empty, Arg::Empty, Arg::Label(start_label
    ));
20 self.ir.emit(Op::Label, Arg::Empty, Arg::Empty, Arg::Label(end_label
    ));
```

此方案确保了控制流转换成线性序列指令,且标号分配全局唯一。

4 详细设计

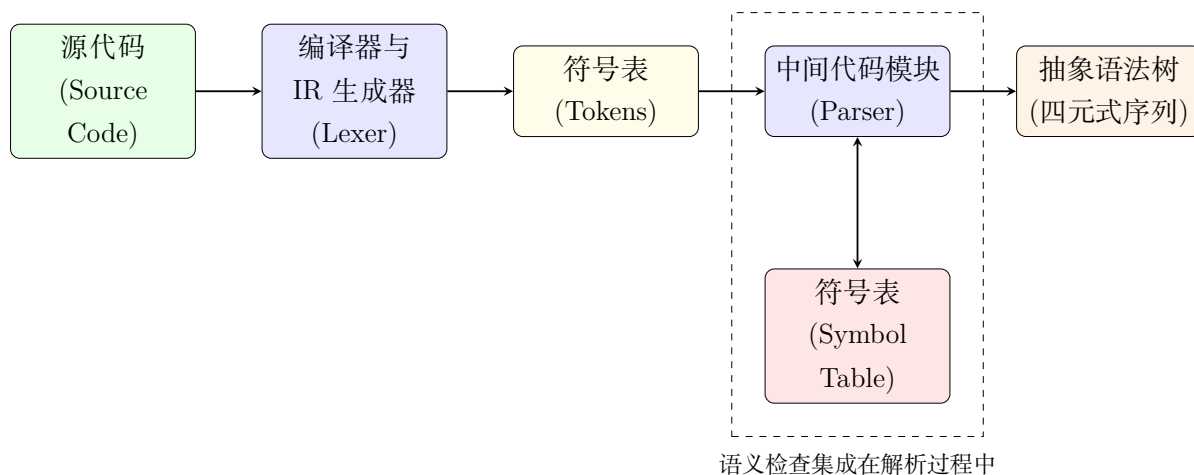


图 1: 系统模块架构图

4.1 编译器与 IR 生成器详细设计

Compiler 结构体负责维护符号表作用域及状态，包含以下字段：

```
1 pub struct Compiler {
2     scopes: Vec<HashMap<String, SymbolInfo>>,
3     ir: IRProgram,
4     temp_count: usize,
5     label_count: usize,
6     errors: Vec<String>,
7     // For break/continue labels: (continue_label, break_label,
8     //   return_type_if_any)
9     loop_stack: Vec<(usize, usize, Option<Type>>>,
10    // For function return type checking
11    current_return_type: Option<Type>,
12 }
```

核心方法 `compile_statement()` 的流程：

1. 采用模式匹配 (Pattern Matching) 判定当前 AST 语句的类型；
2. 遇到变量声明时，向当前作用域 `scopes.last_mut()` 插入带有类型和属性的 `SymbolInfo`；
3. 遇到表达式计算时，递归推导表达式树并实时抛出临时变量，检查类型一致性；
4. 对于控制流语句（如 `While` 或 `If`），申请全局标签，发射 `JUMP_IF_FALSE` 指令控制跳转。

4.2 中间代码模块详细设计

IRProgram 与 Quadruple 结构体包含以下定义：

```
1 #[derive(Debug, Clone)]
2 pub struct Quadruple {
3     pub op: Op,
4     pub arg1: Arg,
5     pub arg2: Arg,
6     pub result: Arg,
7 }
```

```
1 #[derive(Debug)]
2 pub struct IRProgram {
3     pub instructions: Vec<Quadruple>,
4 }
```

核心指令系统覆盖如下操作：

表 2: IR 指令集

指令枚举	说明
Add, Sub, Mul, ...	算术与关系运算
Assign	赋值
Jump	无条件跳转至某标号
JumpIfFalse	当条件伪时跳跃
Label	标记指令块的首地址
Param, Call, Return	函数调用相关操作
LoadArray, StoreArray	数组元素的读取与写入

4.3 四元式序列节点设计

四元式序列的顶层结构为 Program, 包含多个 Declaration (目前仅支持函数声明)。函数声明包含函数名、参数列表、可选返回类型和函数体 (Block)。

语句类型使用 Rust 枚举表示, 包含: Empty, ExprStmt, Return, VarDecl, Assign, VarDeclAssign, If, While, For, Loop, Break, Continue。

表达式类型同样使用枚举, 包含: IntLiteral, Ident, BinaryOp, UnaryDeref, Ref, MutRef, Call, Index, TupleIndex, ArrayLiteral, TupleLiteral, Range, ExprBlock, IfExpr, LoopExpr, Paren。

4.4 错误处理

分析过程中遇到语义错误时，通过调用 `self.error()` 记录并汇总抛出。例如，针对文件 `tests/test_semantic_errors.rs.txt` 的测试，编译器能精确定位以下问题：

- **未声明与作用域越界**：左值变量 'a' 未声明
- **可变性冲突**：不可变变量 'a' 不允许被二次赋值
- **类型一致性约束**：赋值类型不匹配：左值 `I32`，右值 `Array(I32, 3)`
- **返回类型约束**：返回语句类型空和函数声明类型 `I32` 不一致
- **控制流结构约束**：`break` 必须出现在循环体内

5 测试与验证

5.1 测试用例设计

为了充分展示大作业 2 中编译器检查错误的能力和代码生成的能力，我们设计了两组测试用例：

1. **功能测试集 (内置程序 1.1 – 9.3)**：涵盖赋值、复杂控制流 (嵌套 `if/while/for/loop`)、解引用与数组运算。主要用于验证 IR 生成模块 (四元式) 的跳转逻辑是否符合控制流原语。
2. **语义错误测试集**：我们在 `test_semantic_errors.rs.txt` 中构造了 7 种静态语义违规，用于检验类型系统和作用域检查器的鲁棒性。

5.2 语义错误诊断结果示例

对于未声明变量和类型不匹配，编译器输出了精准的诊断结果：

```
1 [Semantic Error] 左值变量 'a' 未声明
2 [Semantic Error] 赋值类型不一致：左值 I32，右值 Array(I32, 3)
3 [Semantic Error] 不可变变量 'a' 不允许被二次赋值
4 [Semantic Error] 二元运算类型不匹配：I32 + Tuple([I32, I32])
5 [Semantic Error] break 必须出现在循环体内
6 [Semantic Error] continue 必须出现在循环体内
7 [Semantic Error] 返回语句类型空和函数声明类型 I32 不一致
```

5.3 中间代码（四元式）生成示例

对于 while 循环和判断嵌套的 program_5_1:

```
1 fn program_5_1(mut n:i32) {
2     while n>0 {
3         n=n-1;
4     }
5 }
```

编译器生成的四元式序列（严格控制流标号与临时变量分配）:

```
1 program_5_1:
2   84: (  PARAM      ,      -      ,      -      ,      n      )
3 L27:
4   86: (    GT      ,      n      ,      0      ,      t13      )
5   87: ( JUMP_IF_FALSE,      t13      ,      -      ,      L28      )
6   88: (    SUB      ,      n      ,      1      ,      t14      )
7   89: (  ASSIGN    ,      t14      ,      -      ,      n      )
8   90: (    JUMP     ,      -      ,      -      ,      L27      )
9 L28:
10  92: (  RETURN    ,      -      ,      -      ,      -      )
```

可以清晰看出，条件 $n>0$ 对应运算 GT，其假值使得程序跳向循环出口标号 L28；而循环体末尾通过无条件 JUMP 回到标号 L27。这完美还原了语义。

6 环境配置与运行指南

6.1 环境配置

本项目使用 Rust 编写，编译和运行需要安装 **Rust 工具链** (1.70+ 版本推荐)。安装方法：

```
1 curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

6.2 编译与测试

在项目根目录 (2)rust_compiler/ 下执行以下命令：

1. **编译**：运行 `cargo build --release`。
2. **运行内置测试**：运行 `cargo run --release`。
3. **测试完整用例**：运行 `cargo run --release -- tests/test_all.rs.txt`。

4. **测试语义错误拦截**: 运行 `cargo run --release -- tests/test_semantic_errors.rs.txt`, 预期会输出由于违反语义检查而导致的错误。

7 心得与思考

7.1 关于 LR(1) 无法处理的情况

PPT 中提到“本规则中存在极个别的羁绊会使得 LR(1) 无法处理”。一个典型的例子是**表达式块**（规则 7.0/7.1）与**普通语句块**之间的歧义。当解析器遇到 `{` 时，无法仅通过有限的前瞻确定这是一个语句块还是一个可以作为表达式的函数表达式块。

在递归下降分析器中，我们可以通过上下文信息（调用者是语句位置还是表达式位置）来区分这种情况，但在 LR(1) 分析器中，这种依赖上下文的消歧较难实现。

7.2 1.4 形参列表的语言

形参列表 $\langle \text{形参列表} \rangle \rightarrow \langle \text{形参} \rangle | \langle \text{形参} \rangle , \langle \text{形参列表} \rangle$ ， $\langle \text{形参列表} \rangle$ 可以识别**至少一个形参**的列表，产生式描述了以逗号分隔的形参序列。结合 $\langle \text{形参列表} \rangle \rightarrow \text{空}$ 的规则，完整支持零个或多个形参的情况。从语言层面看，这是一个正则语言，可用正则表达式描述为 $(\text{形参}(, \text{形参})^*)?$ 。

7.3 9.1 元组 vs 8.1 数组的产生式差异

元组类型比数组类型多了几条产生式，原因在于：

1. **数组**的所有元素类型相同，因此类型声明只需 $[T; N]$ ，一个类型加一个长度即可；
2. **元组**的每个位置可以有不同的类型，需要显式列出每个元素的类型，因此需要 $\langle \text{类型列表} \rangle$ 相关的产生式。
3. 此外，元组还需要区分 (T) 和 $(T,)$ 的情况——前者是括号括起来的类型，后者是单元素元组。

8 参考资料

1. 陈火旺. 程序设计语言编译原理（第 3 版）. 国防工业出版社.
2. Alfred V. Aho et al. Compilers: Principles, Techniques and Tools (Second Edition). 赵建华等译. 机械工业出版社.
3. The Rust Reference. <https://doc.rust-lang.org/reference/>

4. Rust By Example. <https://doc.rust-lang.org/rust-by-example/>
5. Thorsten Ball. *Writing An Interpreter In Go*. 2016.